

Web Application Penetration Testing Report

Prepared for: [NetPoint Internet Solutions]

Prepared by: [Belal Mohamed Fathy]

Date: [12/9/2026]

Classification: CONFIDENTIAL

Table of Contents

1. Executive Summary

2. Scope and Methodology

3. Findings Summary by OWASP Category

4. Detailed Findings

1-env panel Leaked API Data

2-leaked admin panel

3-XSS on search In Products

4-XSS In Products Reviews

5-Weak Password Policy

6- IDOR – Unauthorized Access to Other Users' Orders

7-SQL Injection – Product Category Parameter

5. Risk Ratings

6. Remediation Recommendations

1. Executive Summary

A web application penetration test was conducted against **NetPoint Store**, hosted at <http://localhost:9000>, on behalf of **NetPoint Internet Solutions**. The assessment was performed by **Mostafa Ahmed**.

The assessment evaluated the application across key security areas, including **authentication, authorization, input validation, session management, business logic, API security, and client-side security controls**. Testing was conducted in accordance with the **OWASP Web Security Testing Guide (WSTG) v4** and the **OWASP Top 10 (2021)** framework.

The overall security posture of the application was assessed as **High Risk**. A total of **7 vulnerabilities** were identified across **3 OWASP Top 10 categories**, including **Broken Access Control, Injection, and Security Misconfiguration**.

The identified vulnerabilities may allow unauthorized access to sensitive functionality or data, manipulation of application behavior, and exposure of security-sensitive information. These findings should be prioritized according to their severity and potential business impact, with appropriate remediation measures implemented to strengthen the application's overall security posture.

2. Scope and Methodology

Target URL

- **Target Application:** NetPoint Store
- **Target URL:** http://localhost:9000

Authenticated Roles Tested

- Unauthenticated User
- Standard User
- Administrator

Excluded Areas

No specific areas were excluded from the assessment.

Testing Areas

The assessment focused on the following security areas, aligned with the **OWASP Top 10 (2021)** and relevant web application security testing practices:

- **Injection**
- **Broken Access Control**
- **Security Misconfiguration**
- **Identification and Authentication Failures**
- **Cross-Site Scripting (XSS)**
- **Insecure Direct Object References (IDOR)**

Tools Used

The following tools were used during the assessment:

- **Burp Suite**
- **Web Browser Developer Tools**
- **Web Browser**

Methodology

Testing was conducted manually against the target web application using a structured web application penetration testing methodology. The assessment included **reconnaissance, authentication and authorization testing, input validation testing, access control testing, session and application behavior analysis, and manual verification of identified vulnerabilities.**

Each identified vulnerability was independently validated using controlled test cases. Relevant **HTTP requests and responses, technical evidence, potential impact, and remediation recommendations** were documented for each finding to ensure accurate assessment and reproducibility.

3. Findings Summary by OWASP Category

ID	Finding Title	severity
1	env panel Leaked API Data	High
2	leaked admin panel	High
3	XSS on search In Products	Medium
4	XSS In Products Reviews	Medium
5	Weak Password Policy	Medium
6	IDOR – Unauthorized Access to Other Users' Orders	Medium
7	SQL Injection – Product Category Parameter	High

4. Detailed Findings

4.1: [env panel Leaked API Data]

vulnerability Type: [Information Disclosure]

ID : 1

Severity: High

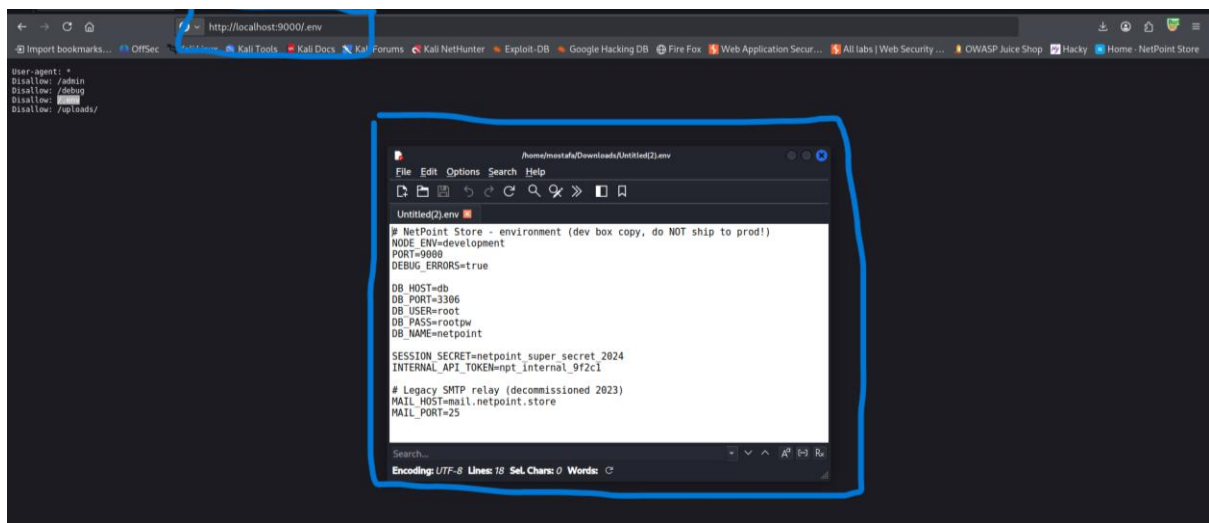
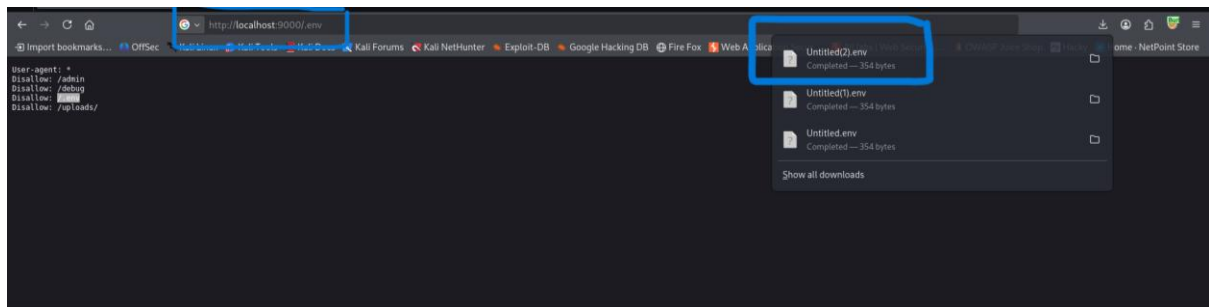
CVSS 3.1 Score: [7.5] — [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N]

Affected URL: <http://localhost:9000/.env>

Description: The .env configuration file is publicly accessible through the web server, allowing unauthenticated users to retrieve sensitive application configuration data. The exposed information may include database credentials, session secrets, API tokens, and other security-sensitive values. The file can be accessed directly without authentication or any user interaction.

Proof of Concept: During reconnaissance, I accessed /robots.txt and identified a reference to the /.env path. I then accessed the exposed path directly through the browser, which resulted in the server making the environment configuration file available for download. After reviewing the downloaded file, sensitive API-related information and credentials were identified.

Impact: The exposed .env file reveals sensitive application configuration and authentication credentials. An attacker could potentially use the disclosed information to gain unauthorized access to internal services, databases, APIs, or other application functionality, depending on the privileges associated with the exposed credentials. The primary demonstrated impact is the loss of confidentiality of sensitive application information.



Recommendation: Restrict public access to the `.env` file and ensure that sensitive configuration files and credentials are stored outside the web-accessible directory. Configure the web server to deny requests for `.env` and other sensitive configuration files. Additionally, immediately rotate any exposed credentials, API keys, and authentication tokens to prevent potential unauthorized use.

.

4.2: [leaked admin panel]

vulnerability Type: [Broken Access Control]

ID : 2

Severity: High

CVSS 3.1 Score: [8.1] — [CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N]

Affected URL: <http://localhost:9000/admin>

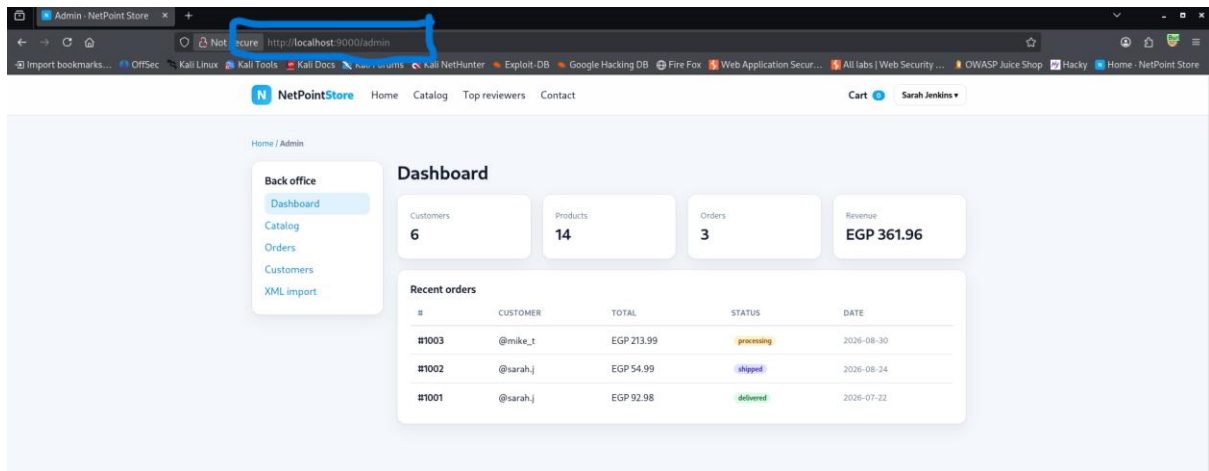
Description: The application exposes an administrative panel that is accessible to a regular authenticated user through the /admin path. The panel provides privileged administrative functionality, including customer and order management and the ability to modify user permissions.

Access to these functions is not properly restricted based on user privileges, allowing a low-privileged authenticated user to access administrative capabilities beyond their intended authorization level. This represents a Broken Access Control vulnerability.

Impact: A low-privileged authenticated user could gain unauthorized access to administrative functions, including modifying user permissions and managing or deleting customer and order data. Successful exploitation could result in unauthorized modification or deletion of application data, potentially compromising the integrity and confidentiality of sensitive information and administrative functionality.

Proof of Concept: During reconnaissance, I accessed the /robots.txt file and identified the /admin path. After authenticating with a standard user account, I navigated directly to the identified administrative endpoint and successfully accessed the Admin Panel.

The panel exposed privileged functionality that should be restricted to administrators, including the ability to modify user permissions and manage customer and order data. This confirmed that proper role-based access controls were not enforced.



Recommendation:

Restrict access to the Admin Panel to authorized administrator accounts only. Implement proper server-side role-based access control (RBAC) to ensure that regular users cannot access administrative endpoints or perform privileged actions, such as modifying user permissions or creating, modifying, or deleting customer and order data.

Access controls should be enforced on the server side for every administrative function rather than relying solely on hidden UI elements or client-side restrictions.

4.3: [XSS on search In Products]

vulnerability Type: [Cross Site Scripting]

ID : 3

Severity: [Medium]

CVSS 3.1 Score: [5.4] — [CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:L/A:N]

Affected URL: <http://localhost:9000/products?search>

parameter : search

Description: The application's search functionality does not properly validate, sanitize, or encode user-supplied input before displaying it in the response. As a result, a malicious JavaScript payload injected into the search parameter is reflected in the page and executed within the victim's browser.

This vulnerability is classified as Reflected Cross-Site Scripting (XSS) and could allow an attacker to execute arbitrary client-side scripts in the context of a user's session.

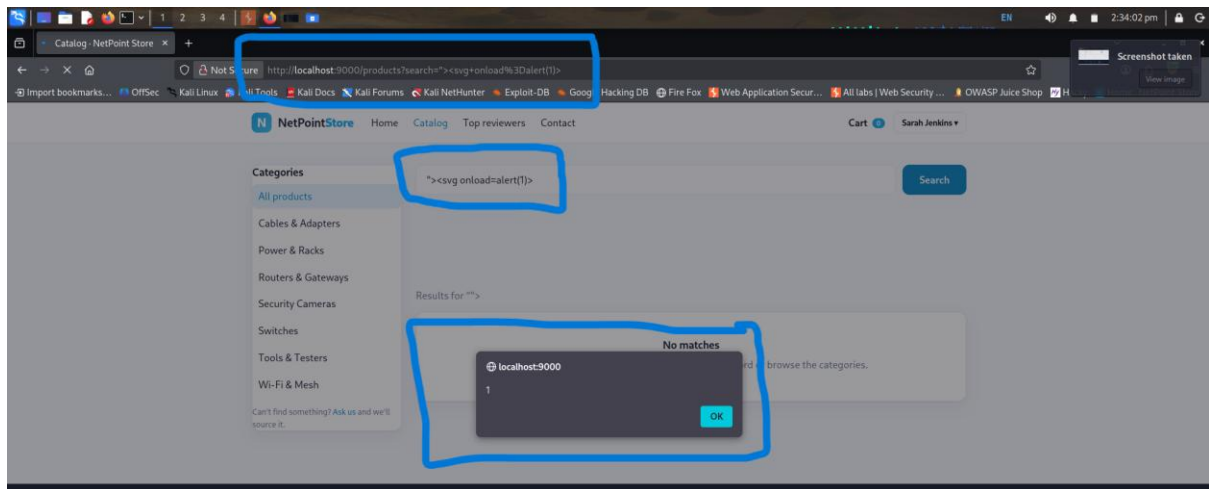
Impact: An attacker could craft a malicious URL containing a JavaScript payload and distribute it to a targeted user. When the victim accesses the URL, the payload may execute within the context of the vulnerable application.

Successful exploitation could allow the attacker to perform unauthorized actions on behalf of the victim or access sensitive information available to the vulnerable page, depending on the victim's privileges and the application's security controls.

Proof of Concept : I navigated to the /products page and entered the following XSS payload into the search field:

"><svg onload=alert(1)>

After submitting the search query, the injected payload was reflected in the application response and executed by the browser, triggering the JavaScript alert. This behavior confirms the presence of a Reflected Cross-Site Scripting (XSS) vulnerability in the search functionality.



Recommendation:

Properly validate and sanitize all user-supplied search input before processing or displaying it. Apply appropriate context-aware output encoding to prevent injected scripts from being interpreted as executable code by the browser.

Additionally, implement a strong Content Security Policy (CSP) as a defense-in-depth measure to reduce the potential impact of XSS vulnerabilities.

4.4: [XSS In Products Reviews]

vulnerability Type: [Cross Site Scripting]

ID : 4

Severity: [Medium]

CVSS 3.1 Score: [5.4] — [CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:L/A:N]

Affected URL: <http://localhost:9000/products/<product-id>>

Parameter: review

Description: The product review functionality does not properly validate, sanitize, or encode user-supplied content before storing and displaying it. As a result, an attacker can submit a malicious JavaScript payload as part of a product review, which is stored by the application and executed in the browser when other users view the affected product page.

This vulnerability is classified as Stored Cross-Site Scripting (XSS) and may allow malicious scripts to execute within the security context of users viewing the compromised product page.

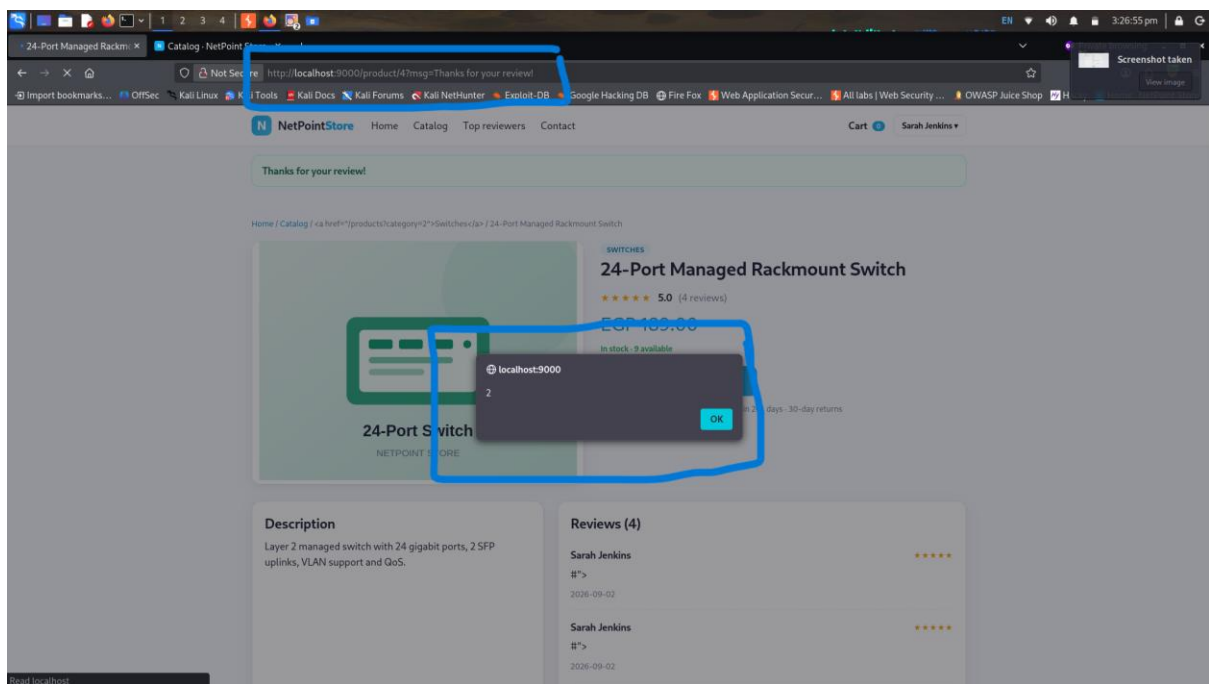
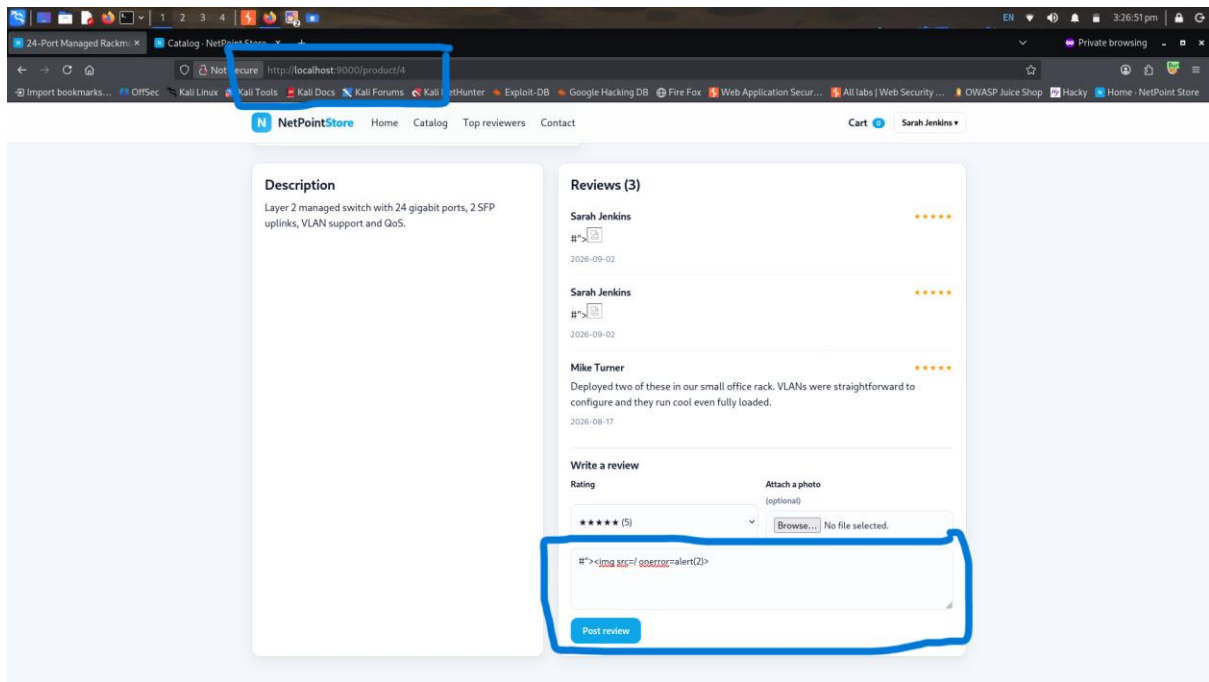
Impact: An attacker can inject malicious JavaScript into a product review, which is then stored and executed in the browsers of users who view the affected review. Successful exploitation may allow unauthorized actions to be performed within the victim's session or enable access to sensitive information available to the vulnerable page, depending on the user's privileges and application security controls.

Proof of Concept: I authenticated using a standard user account, navigated to a product page, and submitted a review containing the following XSS payload:

"><svg onload=alert(2)>

The review was successfully stored by the application. When the affected product page was subsequently accessed, the injected payload was executed in the browser, triggering the JavaScript alert.

This behavior confirms the presence of a Stored Cross-Site Scripting (XSS) vulnerability in the product review functionality.



Recommendation Properly validate and sanitize all user-submitted review content before storing and displaying it. Apply context-aware output encoding to ensure user-supplied content is treated as data rather than executable code.

Additionally, implement a strong Content Security Policy (CSP) as a defense-in-depth measure to reduce the risk and potential impact of Cross-Site Scripting vulnerabilities.

4.5: [Weak Password Policy]

vulnerability Type: [Security Misconfiguration]

ID: 5

Severity: [Medium]

CVSS 3.1 Score: [4.8] — [CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:L/A:N]

Affected URL: <http://localhost:9000/register>

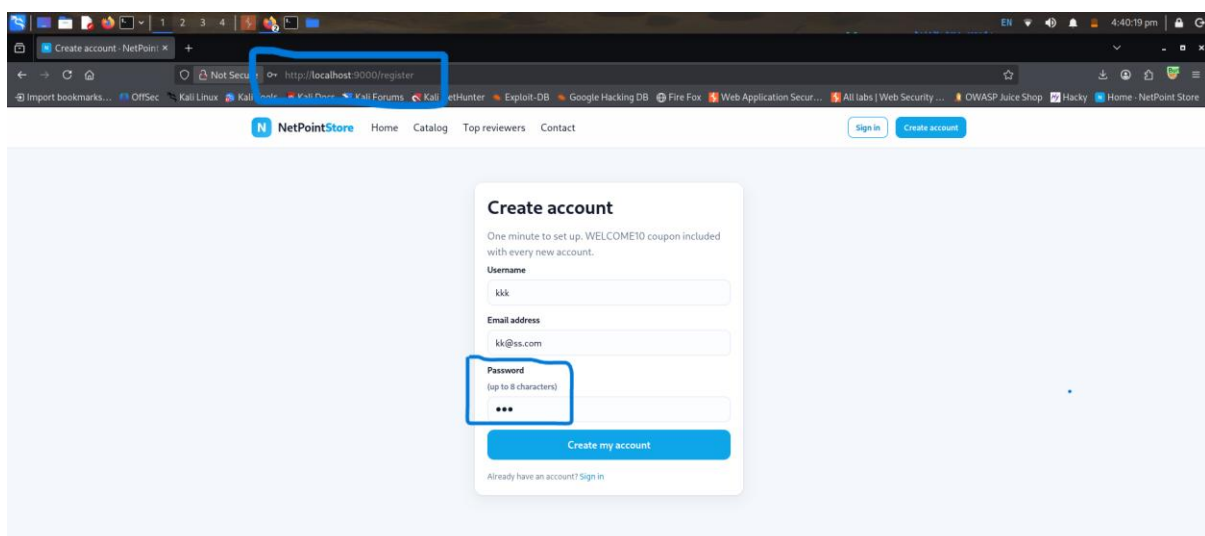
Parameter: password

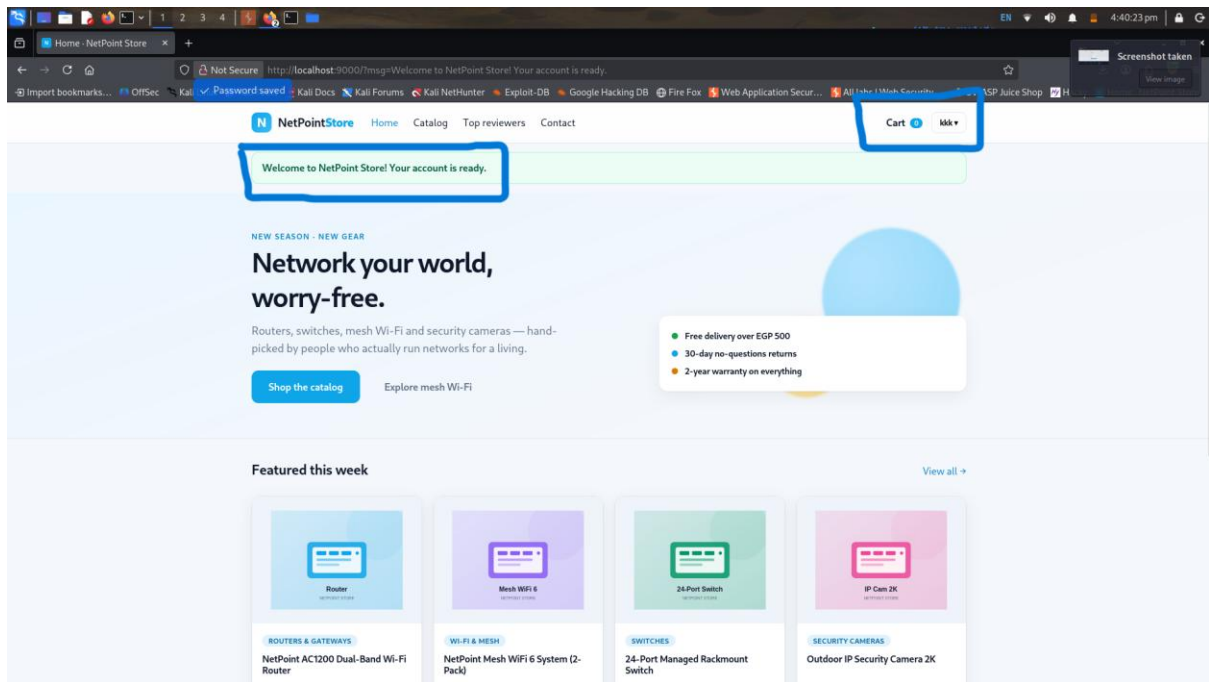
Description: The account registration functionality does not enforce an adequate minimum password length. Users are able to create accounts using extremely short passwords, including passwords consisting of only three characters.

The lack of an effective password policy reduces account security and increases the risk of password guessing, brute-force attacks, and unauthorized account access.

Impact: Weak password requirements increase the likelihood that user accounts can be compromised through password guessing, brute-force attacks, or the use of commonly known passwords. Successful exploitation could result in unauthorized access to user accounts and the sensitive information or functionality associated with them.

Proof of Concept: I navigated to the account registration page and attempted to create a new user account using a three-character password. The application accepted the password and successfully created the account without enforcing a minimum password length requirement. This confirms that the application's password policy does not enforce an adequate minimum password length.





Recommendation: Enforce a minimum password length of at least 8 characters and implement a stronger password policy during account registration. Password requirements should follow current security best practices and discourage the use of weak or commonly compromised passwords.

4.6: [IDOR – Unauthorized Access to Other Users' Orders]

vulnerability Type: [Broken Access Control – IDOR]

ID: 6

Severity: [Medium]

CVSS 3.1 Score: [6.5] —[CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N]

Affected URL: http://localhost:9000/orders/<order_id>r

Parameter: order_id

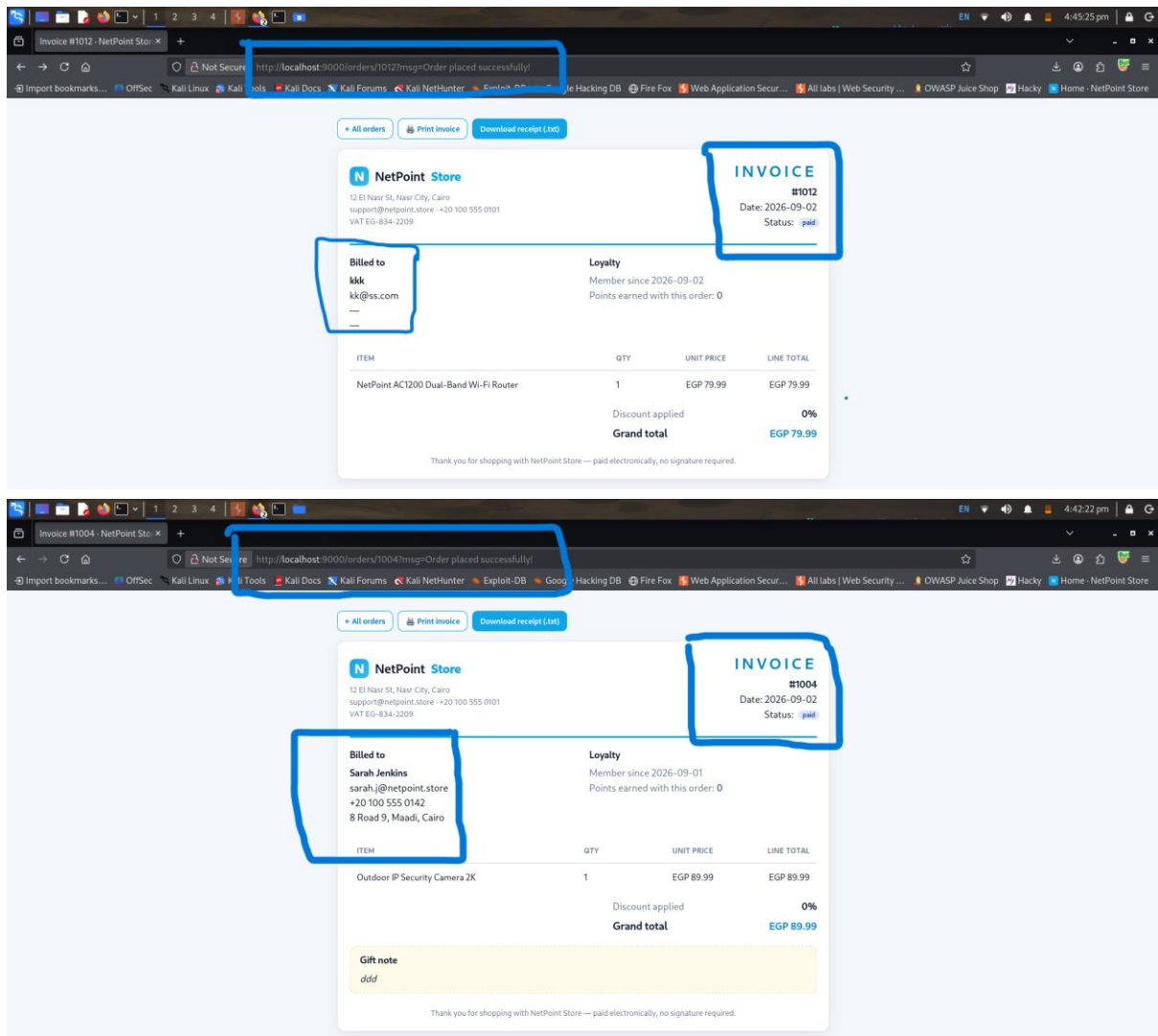
Description: The application does not properly enforce authorization checks when users access order records. By modifying the order ID within the request, an authenticated user can access order details belonging to other users.

This vulnerability represents an Insecure Direct Object Reference (IDOR) and indicates insufficient server-side access control over sensitive user-specific resources.

Impact: An attacker can modify the order ID in a request to access order information belonging to other users. Successful exploitation may result in the unauthorized disclosure of sensitive customer and order data, compromising the confidentiality and privacy of user information.

Proof of Concept: I accessed an order through the checkout/order page using a standard authenticated user account and modified the order ID in the URL. The application returned the details of another user's order without verifying whether the authenticated user was authorized to access the requested resource.

This confirms the presence of an Insecure Direct Object Reference (IDOR) vulnerability caused by insufficient server-side authorization checks.



Recommendation: Implement robust server-side authorization checks for every order-related request. Before returning order details, verify that the requested order belongs to the currently authenticated user or that the user has an explicitly authorized administrative role. Do not rely solely on client-side controls or user-supplied order IDs to enforce access restrictions.

4.7: [SQL Injection – Product Category Parameter]

vulnerability Type: [SQL Injection]

ID : 7

Severity: [High]

CVSS 3.1 Score: [7.5] - [CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N]

Affected URL: <http://localhost:9000/products?category=<value>>

Parameter: category

Description: The product category functionality is vulnerable to SQL Injection because user-supplied input is incorporated into database queries without adequate validation or parameterization. By submitting specially crafted input, an attacker can trigger SQL errors and potentially alter the logic of the underlying database query.

This vulnerability indicates insufficient input handling and could allow an attacker to interact with the application's database beyond the intended functionality.

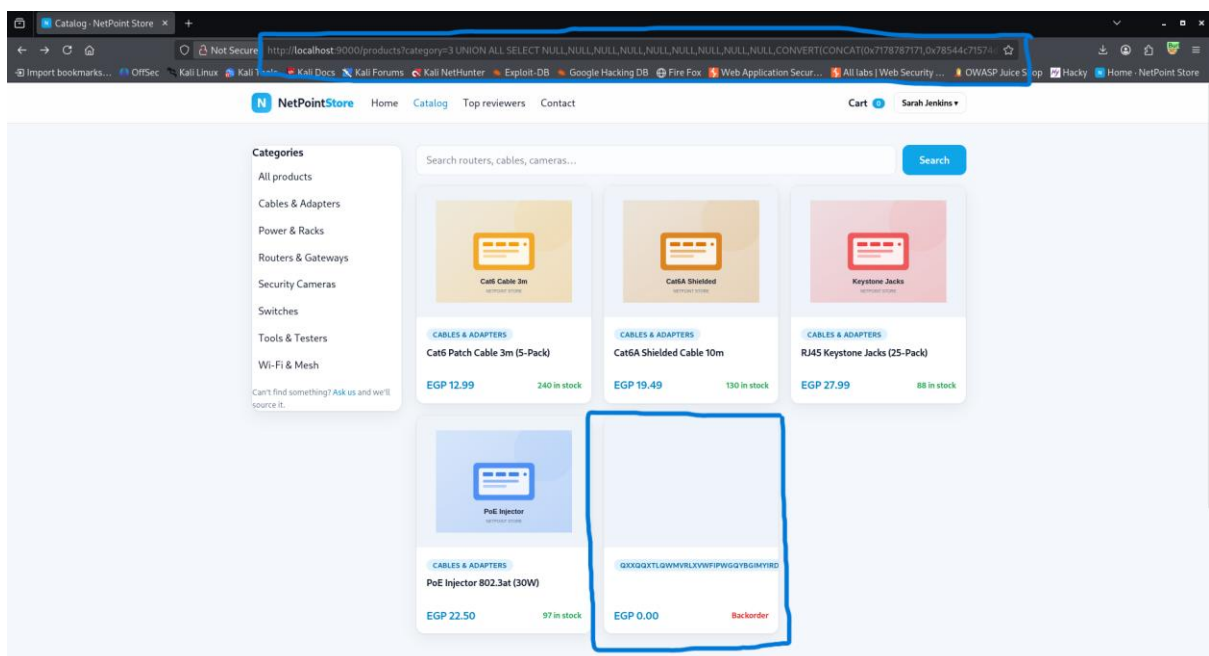
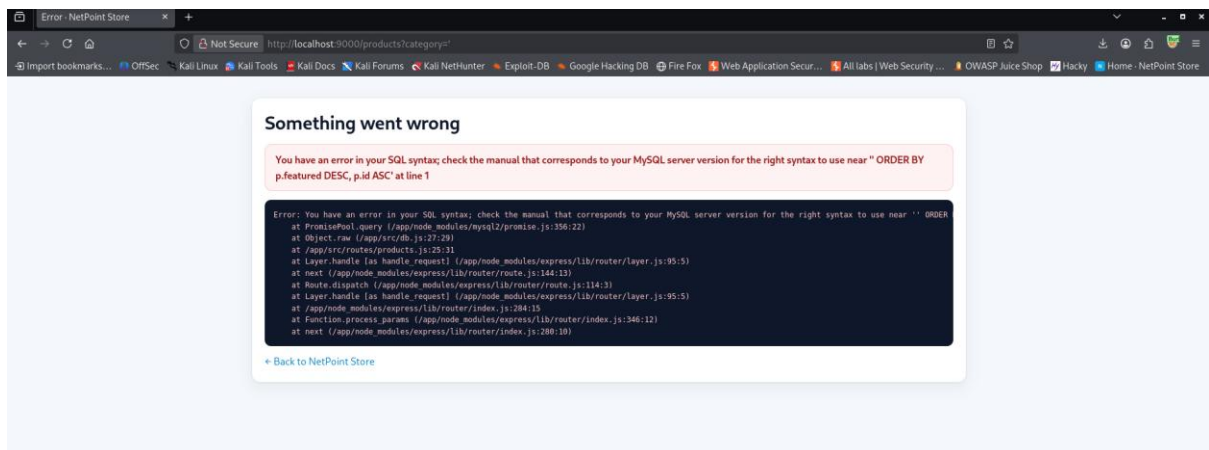
Impact: An attacker may be able to manipulate the application's database queries and potentially access or modify unauthorized database information. In addition, detailed SQL error messages may expose internal database structure, query information, or application details, providing valuable information that could assist in further attacks against the application.

Proof of Concept I navigated to the product category functionality and modified the category parameter with SQL injection test input. The application returned a detailed SQL error message, indicating that the supplied input was being processed directly by the underlying database query.

Further controlled testing demonstrated that the query behavior could be manipulated using a UNION-based SQL injection technique, confirming that the category parameter is vulnerable to SQL Injection.

Payload:

[Insert the tested SQL injection payload here]



Recommendation: Use parameterized queries or prepared statements for all database operations to ensure that user-supplied input cannot alter the structure of SQL queries. Properly validate and constrain input based on the expected data type and format. Additionally, disable detailed SQL error messages in production and return generic error responses to users. Detailed database errors should be logged securely on the server for troubleshooting without exposing sensitive implementation details to clients.

5. Risk Ratings

4. Risk Rating Classification

The severity of each identified vulnerability was assessed based on its potential impact, exploitability, and the level of access or damage that could result from successful exploitation. The following severity levels were used:

Critical — 9.0–10.0

Vulnerabilities that pose an immediate and severe risk to the application or underlying infrastructure. Successful exploitation may result in complete system compromise, remote code execution, authentication bypass, or extensive exposure or compromise of sensitive data.

High — 7.0–8.9

Vulnerabilities that could result in significant unauthorized access, sensitive information disclosure, privilege escalation, or compromise of critical application functionality.

Medium — 4.0–6.9

Vulnerabilities with a moderate security impact that may allow limited unauthorized access, information disclosure, or manipulation of application functionality. Examples may include XSS, IDOR, or SQL Injection with limited demonstrated impact.

Low — 0.1–3.9

Vulnerabilities with limited security impact that generally require additional conditions or successful exploitation of other weaknesses to cause significant harm. Examples include weak security configurations or insufficient password requirements.

6. Remediation Recommendations.

6. Remediation Recommendations

The following recommendations are prioritized to address the vulnerabilities identified during the assessment and strengthen the overall security posture of the application:

- 1. Prevent SQL Injection**
Use parameterized queries or prepared statements for all database operations. Properly validate and constrain user-supplied input before processing it.
- 2. Implement Role-Based Access Control (RBAC)**
Enforce server-side authorization controls to ensure that administrative functionality is accessible only to authorized administrator accounts.
- 3. Strengthen Object-Level Authorization**
Verify that authenticated users are authorized to access each requested resource. Ensure that users cannot access or manipulate other users' data by modifying object identifiers, such as order IDs.
- 4. Prevent Cross-Site Scripting (XSS)**
Properly validate and sanitize user-supplied input and apply context-aware output encoding before displaying user-controlled content. Implement a strong Content Security Policy (CSP) as an additional security measure.
- 5. Protect Sensitive Configuration Files and Credentials**
Restrict public access to sensitive files such as .env and store configuration secrets outside web-accessible directories. Immediately rotate any credentials, API keys, or tokens that may have been exposed.
- 6. Strengthen Password Policies**
Enforce an appropriate minimum password length and stronger password requirements during account registration. Consider implementing protections against commonly used and compromised passwords.
- 7. Prevent Information Disclosure Through Error Messages**
Disable detailed SQL and database error messages in production environments. Return generic error messages to users while securely logging detailed technical information on the server for troubleshooting and monitoring.